



SPA con React

6. Publicación



- **Ejecución en modo “desarrollo”**
 - Hasta ahora hemos usado **React Router** para ejecutar aplicaciones durante el desarrollo
 - **LiveReload**: Guardar fichero y actualización del navegador
 - Código optimizado para **depuración y recarga** rápida

```
$ npm run dev
```

- Ejecución en modo “producción”
 - Las aplicaciones se preparan para que puedan ser **usadas** por los usuarios
 - Se optimizan para consumir **pocos recursos** (CPU, ancho de banda...)
 - A esta fase se la denomina **construcción** (*build*)
 - Posteriormente se **publican** (*publish*) o **despliegan** (*deploy*) en un **servidor** o **servicio**

- Construcción y publicación de aplicaciones web SPA
 - Las aplicaciones SPA tienen parte **frontend** (**cliente**) y parte **backend** (**servidor, API REST**)
 - **Frontend:** se implementa con React (o con cualquier otro framework)
 - **Backend:** se implementa con cualquier tecnología como Java, Node, Groovy, PHP, Python

- **Construcción y publicación del backend (API REST)**
 - Se empaqueta dependiendo de la tecnología usada para su creación (Java, PHP, Python, Go, Node...)
 - Es habitual que el paquete nativo se empaquete además usando **Docker** y se ejecute en un **servidor**
 - Ofrece un interfaz de comunicación con el SPA basada en http: API REST, WebSocket, Server Side Events..

- Construcción y publicación del frontend SPA
 - 1) Construcción
 - HTML, CSS, JS, imágenes estáticas, iconos...
 - Optimizados para ocupar el menor espacio posible (**reducir la transferencia**)
 - Se reduce el **tiempo de carga** en el navegador (quitar comentarios, código que no se ejecuta...)

- **Construcción y publicación del frontend SPA**
 - 2) Publicación (alternativas):
 - **Servidor HTTP:** Exclusivo para el frontend (NGINX, Apache2, http-serve...)
 - **Backend:** El backend de la aplicación, que ofrece la API REST, puede usarse para alojar los estáticos del frontend
 - **Servicios de publicación:** CDN, GitHub Pages...

- **Single Page Aplicacion (SPA) vs Server Side Rendering (SSR)**
 - Las aplicaciones **SPA** cargan un HTML sencillo (Loading...) y cuando se ejecuta el JavaScript se muestra el HTML real.
 - **Más tiempo en mostrar la página:** Hay que descargar JS (la primera vez) y ejecutarlo
 - **Complica buscadores:** El primer HTML es incompleto
 - El **Server Side Rendering (SSR)** permite ejecutar el JS en el servidor para generar el HTML “útil” y enviar el JS en segundo plano (para interactividad)

- **Single Page Aplicacion (SPA) vs Server Side Rendering (SSR)**
 - Hace falta desplegar el JavaScript en el servidor (con Node.js)
 - El SSR está muy acoplado al routing y cada framework lo implementa de una forma diferente



NEXT.js



- **Single Page Application (SPA) vs Server Side Rendering (SSR)**
 - Por simplicidad NO vamos a estudiar Server Side Rendering
 - Desplegaremos la aplicación en modo SPA (desactivando el SSR)



NEXT.JS



- **1) Desactivamos SSR**
 - Desactivamos SSR en `react-router.config.ts`

```
import type { Config } from "@react-router/dev/config";  
  
export default {  
  // Config options...  
  // Server-side render by default, to enable SPA mode set this to `false`  
  ssr: false,  
} satisfies Config;
```

- **1) Desactivamos SSR**
 - Se añade el HTML que se mostrará mientras se carga la SPA en `root.tsx`

```
export function HydrateFallback() {  
  return <p>Loading...</p>;  
}
```

- **2) Construcción**

- Se ejecuta el comando

```
$ npm run build
```

- Los ficheros que deben publicarse en la raíz del servidor http se generan en la carpeta

`/build/client`

<https://reactrouter.com/api/other-api/dev#react-router-build>

- 3) Publicación en un servidor web
 - En SPA la rutas a la API REST tiene que ser absolutas

```
http://127.0.0.1:8080/api/books
```

- Se usar el servidor **http-serve** con NPM

```
$ sudo npm install -g http-serve
```

```
$ http-serve -p 8081 build/client
```

<http://127.0.0.1:8081>

- 4) Publicación en la app con la API REST
- SpringBoot
 - En la SPA las rutas a la API REST son relativas

```
/api/books
```

- Los ficheros estáticos se copian en

```
src/main/resources/static
```

- **Aplicaciones SPA con Router**
 - Las URLs de las páginas de la app no son conocidas por el servidor
 - Si ponemos esa URL en el navegador, el servidor buscará un fichero o un Controller
 - Como no la encuentra, devolverá un error 404

<http://localhost:8080/books>



- **Aplicaciones SPA con Router**
 - Hay que **configurar en el servidor** web para que cuando le soliciten cualquier URL no conocida devuelva el fichero **index.html** de React
 - El código JS de React interpretará esa URL y cargará la página que le corresponde
 - Cada servidor se configura de una forma diferente

<http://localhost:8080/books>



- 4) Publicación en la app con la API REST
- Spring Boot

```
package com.github.codeurjc.books;  
import org.springframework.stereotype.Controller;  
import org.springframework.web.bind.annotation.GetMapping;  
  
@Controller  
public class SPAController {  
  
    @GetMapping("/{path:[^\\.]*}")  
    public String redirect() {  
        return "forward:/";  
    }  
}
```

Este método se ejecuta cuando se pida una ruta sin puntos que no esté especificada en otro Controller

Se devuelve el recurso en /
Corresponde al index.html de la app

- 4) Publicación en la app con la API REST
 - Node.js con Express
 - Se configura la app para servir la carpeta public

public

- Los ficheros estáticos se copian en la carpeta public

```
app.use(express.static('public'));
```

- 4) Publicación en la app con la API REST
- Node.js con Express
 - Se configura la app para devolver index.html si no hay un recurso en public para esa URL

```
const app = express();  
  
...  
app.use(express.static('public'));  
app.get('*', (req, res) => res.sendFile('index.html', { root: 'public' }));  
  
...
```

- 4) Publicación en la app con la API REST
 - Script automatizado (linux)

```
cd ./frontend && npm run build
cd ..

mkdir -p ./backend/java/src/main/resources/static/
rm -rf ./backend/java/src/main/resources/static/*
cp -r ./frontend/build/client/* ./backend/java/src/main/resources/static/

mkdir -p ./backend/node/public/
rm -rf ./backend/node/public/*
cp -r ./frontend/build/client/* ./backend/node/public/
```

- **Aplicación SPA en una ruta específica**
 - Cuando la SPA no está en la raíz / y está alojada en una ruta (p.e. **spa**)
 - Hay que configurar la ruta al construir
 - Hay que copiar los estáticos a esa carpeta
 - Hay que gestionar las URLs no conocidas de forma especial

<http://localhost:8080/spa/>

- Aplicación SPA en una ruta específica
- Construcción

vite.config.ts

```
import { reactRouter } from "@react-router/dev/vite";
import tailwindcss from "@tailwindcss/vite";
import { defineConfig } from "vite";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig(({ mode }) => ({
  base: mode === "production" ? "/spa/" : "/",
  plugins: [tailwindcss(), reactRouter(), tsconfigPaths()],
  server: {
    proxy: {
      "/api": {
        target: "http://localhost:8080/api",
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ""),
      },
    },
  },
}));
```

Se añade esta línea

- Aplicación SPA en una ruta específica
- Construcción

react-router.config.ts

```
import type { Config } from "@react-router/dev/config";  
  
export default {  
  // Config options...  
  // Server-side render by default, to enable SPA mode set this to `false`  
  ssr: false,  
  basename: process.env.NODE_ENV === "production" ? "/spa" : "/",  
} satisfies Config;
```

Se añade esta línea

- Aplicación SPA en una ruta específica
 - Spring Boot
 - Los ficheros se copian a

```
src/main/resources/static/spa
```

Spring Boot

exam38_build_folder

```
@Configuration
public class SpaRoutingConfig implements WebMvcConfigurer {

    private static final String SPA_ROUTE = "/spa";

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {

        registry.addResourceHandler(SPA_ROUTE, SPA_ROUTE + "**")
            .addResourceLocations("classpath:/static" + SPA_ROUTE + "/")
            .resourceChain(true)
            .addResolver(new PathResourceResolver() {
                @Override
                protected Resource getResource(String resourcePath, Resource location) throws IOException {
                    Resource requestedResource = location.createRelative(resourcePath);
                    return requestedResource.exists() && requestedResource.isReadable()
                        ? requestedResource
                        : new ClassPathResource("/static" + SPA_ROUTE + "/index.html");
                }
            });
    }
}
```



Si el recurso no existe,
se devuelve index.html

- Aplicación SPA en una ruta específica
 - Node.js con Express
 - Los ficheros se copian a

public/spa

- Aplicación SPA en una ruta específica
- Node.js con Express
 - Se devuelve index.html si la ruta no es un fichero

```
...  
const app = express();  
  
...  
app.use(express.static('public'));  
  
const SPA_ROUTE = '/spa';  
  
app.use(SPA_ROUTE, express.static(`public${SPA_ROUTE}`));  
app.get(SPA_ROUTE, (req, res) => res.sendFile('index.html', { root: `public${SPA_ROUTE}` }));  
app.get(`${SPA_ROUTE}/*`, (req, res) => res.sendFile('index.html', { root: `public${SPA_ROUTE}` }));
```

- Una app SPA se publica en un **servidor http**
- Servidor **independiente de la API REST**
 - Ruta a API REST **absoluta** en SPA
 - API REST con CORS configurado
- Servidor **de la API REST**
 - Ruta a API REST **relativa** en SPA
 - API REST sin CORS configurado

- Si la app SPA se despliega en una ruta
 - Construcción especial
 - Se copian los estáticos a una carpeta
 - Hay que configurar el servidor http para que al solicitar una ruta que no corresponde con un fichero, se devuelva index.html